Computer Networks - Tutorial sheet 1

Q1. Communication through the OSI model

The OSI model divides communication into seven layers. Data travels down the sender's stack, crosses the medium, and moves up the receiver's stack. Each layer uses the service below it and serves the layer above.

Sender: functions of the seven layers

7. Application: provides network services such as HTTP, email and file transfer. Example: a browser creates an HTTP request.

6. Presentation: handles data representation, encoding, compression and encryption. UTF-8 and encryption libraries perform such functions in practical systems.

5. Session: establishes, manages and ends a dialogue. It may provide synchronization or checkpoints. Internet applications often implement these functions themselves.

4. Transport: provides process-to-process delivery using ports. TCP divides a byte stream into segments and uses sequence numbers, ACKs and retransmission for reliable, ordered delivery.

3. Network: adds logical addresses and supports routing between networks. Routers forward IP packets toward the destination using routing information.

2. Data link: creates frames for one link, using MAC addresses and an error-detection field such as Ethernet FCS. Switches normally forward frames by destination MAC address.

1. Physical: represents bits as electrical, optical or radio signals and carries them through the medium.

Encapsulation and an example

Encapsulation means adding protocol control information as data moves downward. The data from one layer becomes the payload of the next. Ethernet also adds a trailer.

```
Application data
 [ TCP header | application data ]
  [ IP header | TCP header | application data ]
   [ MAC header | IP | TCP | data | FCS ]  ->  bits on the link
```

For an HTTP file request over TCP, TCP adds ports and sequence information; IP adds the endpoint IP addresses; Ethernet adds local-link MAC addresses. The physical interface sends the resulting bits as signals.

If the server is remote, the first destination MAC belongs to the default gateway. A router removes the incoming frame and creates a new outgoing frame. Link addresses change along the route; endpoint IP addresses normally stay the same when there is no NAT.

Receiver: decapsulation

The receiver recovers bits, checks the Ethernet frame and removes its link header and trailer. IP processes its header and selects TCP. TCP checks, reorders and acknowledges the bytes, removes duplicates, and delivers the stream to the correct application.

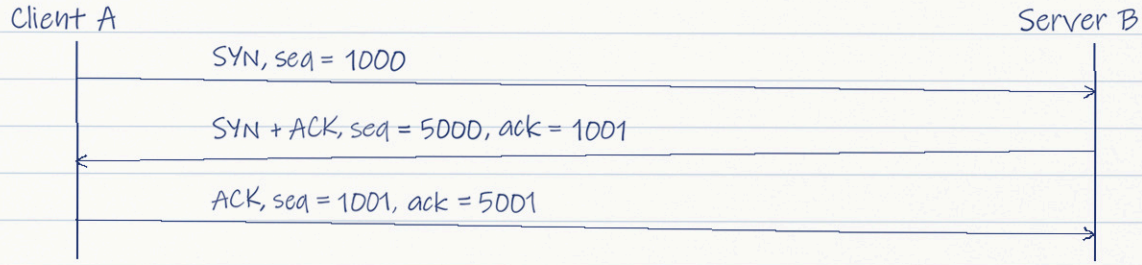
The application interprets the message and displays or stores the file. Thus encapsulation adds control information at the sender, while decapsulation processes and removes it at the receiver.

Tutorial sheet 1

Q7. TCP three-way handshake

TCP has an independent byte sequence in each direction. Both endpoints must learn the other's initial sequence number and confirm that their own number was received. The three-way handshake performs this synchronization.

Let the client choose 1000 and the server choose 5000 as illustrative initial sequence numbers. The server initially listens at its service port. Time moves downward in the diagram.



Step 1: SYN

The client sends SYN = 1, sequence number 1000, and enters SYN-SENT. This announces its initial sequence number and requests a connection. ACK is not set in the ordinary initial SYN.

Step 2: SYN + ACK

The server replies with SYN = 1, ACK = 1, sequence number 5000 and acknowledgment 1001. It enters SYN-RECEIVED.

ACK 1001 confirms the client's SYN because a SYN uses one sequence number. The same segment announces the server's initial number, combining two functions in one message.

Step 3: ACK

The client returns sequence number 1001 and acknowledgment 5001. The client enters ESTABLISHED; the server does so when this acknowledgment arrives.

A pure ACK does not consume a sequence number. Therefore the client's first data can still start at 1001. If it sends 200 bytes, those are bytes 1001-1200, and the server acknowledges 1201 after receiving them in order.

Why three steps are needed

The first message announces the client's sequence number. The second confirms it and announces the server's number. The third confirms that the server's response reached the client.

With only two messages, the server would not know whether its SYN + ACK arrived. An old delayed SYN could otherwise create an apparently established connection without a current client wanting it.

The final ACK ties the exchange to the current sequence numbers and helps prevent old-duplicate confusion. This synchronization is not cryptographic authentication or encryption.

If a message is lost

Lost SYN or SYN + ACK segments are retransmitted after timeout. If the final ACK is lost, the server may repeat SYN + ACK and the client acknowledges it again. A valid ACK carried with later data can also complete establishment.

Options such as MSS, window scaling and SACK permission can be exchanged in SYN segments. The handshake establishes communication state; it does not reserve bandwidth or a dedicated route.

Tutorial sheet 2

Q3. Encoding the bits 1 0 1 1 0 0 1

H means high level and L means low level. Conventions must be stated because NRZI and Manchester can be described using opposite polarity rules.

NRZ-L

Choose 1 = H and 0 = L. Each level remains constant for the full bit interval, without returning to zero between bits.

Levels: H, L, H, H, L, L, H

NRZI

Choose initial level L and a transition at the start of every 1; a 0 leaves the level unchanged.

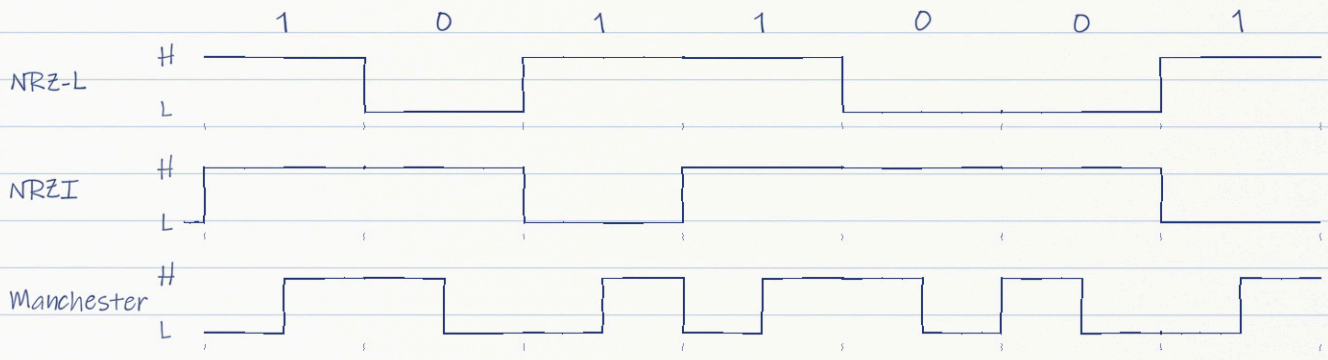
Levels: H, H, L, H, H, H, L

Bit 1 changes L to H; bit 2 stays H; bit 3 changes to L; bit 4 changes to H. Bits 5 and 6 stay H, and bit 7 changes to L.

Manchester

Use the IEEE 802.3 convention: 1 = low then high; 0 = high then low. Every bit has a transition at its midpoint.

Half-bit pairs: LH | HL | LH | LH | HL | HL | LH



Each pair of small ticks encloses one bit interval.

Advantages and limitations

NRZ-L is simple and uses less signaling bandwidth than comparable Manchester coding. Long runs of identical bits give no transitions, making clock recovery difficult. Unbalanced patterns can also cause a DC component or baseline wander.

NRZI identifies changes relative to the previous level, which helps with overall polarity inversion. Under the stated transition-on-1 rule, long runs of zeros still lack transitions. Additional coding or stuffing may be needed.

Manchester is self-clocking because every bit has a midpoint transition. With symmetric levels, each bit is balanced. Its cost is two signal elements per bit and a greater bandwidth requirement.

Extra Manchester transitions can occur at bit boundaries; they are not additional data bits. Decode using the bit timing and midpoint direction.

For data rate R, a bit lasts T = 1/R. Manchester's half-bit interval is T/2, giving a half-bit signaling clock of 2R. The sketches show logical timing only, with time increasing to the right.